

**IMPLEMENTASI DAN ANALISIS
PERBANDINGAN
ALGORITMA PENCARIAN JALUR TERPENDEK
PADA GRAF BERBASIS GUI PYTHON**

LAPORAN TUGAS 1

Oleh:

ARDIAN

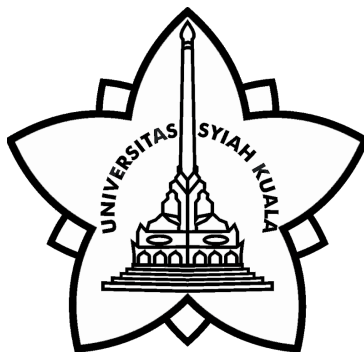
NPM. 260820701100008

MUHAMMAD ICHSAN

NPM. 260820701100007

IRFAN SYAHPUTRA

NPM. 2308107010030



**PROGRAM STUDI S2 - KECERDASAN BUATAN
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS SYIAH KUALA, BANDA ACEH**

2026

1. PENJELASAN PROYEK

1.1 DESKRIPSI UMUM

Pencarian rute terpendek (*shortest path problem*) merupakan salah satu persoalan inti dalam pemodelan graf dan Kecerdasan Artifisial. Persoalan ini berfokus pada penemuan urutan sisi yang menghubungkan simpul awal (*start node*) ke simpul tujuan (*goal node*) sedemikian rupa sehingga akumulasi total bobot sisi bernilai minimum.

Proyek ini merealisasikan sebuah aplikasi perangkat lunak desktop berbasis *Graphical User Interface* (GUI) interaktif menggunakan bahasa pemrograman Python 3. Aplikasi ini dirancang secara khusus untuk memvisualisasikan, mensimulasikan langkah demi langkah (*step-by-step playback*), dan membandingkan performa dua algoritma penelusuran rute terpendek secara langsung dan waktu-nyata (*real-time*).

1.2 ARSITEKTUR PERANGKAT LUNAK DAN KONTRIBUSI ANGGOTA

Aplikasi dibangun secara terpadu melalui integrasi komponen yang dirancang secara modular:

- **Desain Aplikasi dan Sistem Kontrol Playback:** Merancang tata letak antarmuka utama, panel kontrol penelusuran (putar, jeda, maju, mundur, slider progres, penyesuaian kecepatan), serta orkestrasi keadaan pemutaran langkah simulasi graf dan tabel.
- **Mesin Visualisasi Graf dan Efek Animasi Kanvas:** Membangun kanvas interaktif bertema gelap, tata letak posisi simpul otomatis menggunakan algoritma pegas (*Force-directed Spring Layout* Fruchterman–Reingold), perender sisi kurva Bézier kuadratik, efek partikel pergerakan, dan penelusuran pohon pendahulu (*predecessor tree*).
- **Panel Analisis, Integrasi Sistem, dan Benchmark:** Mengembangkan panel kanan yang menyajikan tab ringkasan kartu komparasi kedua algoritma, tabel live simulasi jarak ($d[v]$, $\pi[v]$, status simpul), mesin benchmark statistik 31 iterasi independen, modul ekspor data terstruktur (JSON/teks), dan pengujian otomatis.

Tabel 1. Daftar berkas dan struktur modular program

Berkas	Komponen	Deskripsi Fungsi
MMAIKelompok5.py	Entry Point Utama	Peluncur mandiri aplikasi GUI desktop serta penyedia mode Command Line Interface (<code>-cli</code>).
algorithms.py	Engine Algoritma	Implementasi algoritma Dijkstra (Min-Heap) dan Bellman–Ford beserta pencatat langkah (<i>trace</i>) dan metrik inspeksi.
comparison.py	Engine Benchmark	Mekanisme benchmarking statistik (median, kuartil Q1/Q3, min/max) sebanyak 31 sampel dengan eliminasi bias urutan.
graph_io.py	Parser I/O Graf	Pembaca berkas graf berbasis Abstract Syntax Tree (<code>ast.parse</code>) yang aman dari injeksi eksekusi kode.
graph_view.py	Kanvas Graf GUI	Komponen kanvas graf interaktif (<i>drag</i> , <i>pan</i> , <i>zoom</i>) dengan rendering visual kurva Bézier dan penanda rute akhir.
results_panel.py	Panel Metrik Kanan	Menampilkan kartu komparasi hasil dan tabel perubahan jarak simpul secara dinamis.
playback.py	State Playback	Mengelola sinkronisasi navigasi maju/mundur antara kanvas graf dan tabel live.
layout.py	Tata Letak Pegas	Algoritma otomatis penataan koordinat simpul secara seimbang dan proporsional.

1.3 FITUR UTAMA APLIKASI

1. **Kanvas Graf Interaktif Penuh:** Pengguna dapat menggeser posisi simpul (*drag node*) untuk merapikan representasi visual, menggeser kanvas (*pan*), memperbesar/memperkecil graf (*zoom*), serta mengubah titik awal/tujuan secara cepat lewat klik kanan pada simpul.
2. **Simulasi Langkah Dua Arah (*Bi-directional Step Playback*):** Menghadirkan kontrol putar otomatis, jeda, langkah maju satu per satu, langkah mundur ke kondisi sebelumnya, pengaturan kecepatan ($0.5\times$ hingga $4\times$), serta *scrubber slider* untuk melompat langsung ke langkah ke- N .
3. **Tabel Dinamis & Visualisasi Status:** Memperbarui nilai jarak simpul ($d[v]$) dan simpul pendahulu ($\pi[v]$) secara real-time pada setiap tahap relaksasi sisi.
4. **Benchmark Akurat & Ekspor Multi-Format:** Mengisolasi waktu komputasi mur-

ni dari proses rendering antarmuka dan menyediakan fungsi ekspor hasil perbandingan ke format JSON dan TXT.

2. ALGORITMA DAN LIBRARY YANG DIGUNAKAN

2.1 ALGORITMA PENCARIAN JALUR TERPENDEK

Proyek ini mengimplementasikan dua algoritma pencarian jalur terpendek dengan karakteristik dan paradigma yang berbeda:

2.1.1 Algoritma Dijkstra (Min-Priority Queue)

Algoritma Dijkstra adalah algoritma berbasis pendekatan *greedy* yang memecahkan masalah *Single-Source Shortest Path* pada graf dengan bobot sisi bernilai non-negatif ($w(u, v) \geq 0$).

Pada program ini, algoritma Dijkstra dioptimasi menggunakan:

- **Min-Heap Biner (heapq):** Mengorganisasi simpul aktif dalam antrean prioritas berbasis nilai jarak sementara terkecil, sehingga penarikan simpul terdekat berbiaya waktu logaritmik $O(\log |V|)$.
- **Lazy Duplication:** Menghindari operasi pembaruan kunci (*decrease-key*) yang kompleks dengan cara memasukkan pasangan jarak baru ke antrean, lalu mengabaikan simpul usang yang telah berstatus permanen (*settled*) saat ditarik.
- **Terminasi Dini (Early Goal Termination):** Pencarian segera dihentikan tepat ketika simpul tujuan berhasil ditetapkan sebagai *settled*, tanpa perlu menyelesaikan seluruh sisa simpul dalam graf yang tidak relevan.

2.1.2 Algoritma Bellman–Ford

Algoritma Bellman–Ford adalah algoritma berbasis pemrograman dinamis (*Dynamic Programming*) yang mengevaluasi dan merelaksasi seluruh sisi pada graf secara berulang. Secara teoretis, algoritma ini mampu memproses graf dengan bobot negatif serta mendeteksi adanya siklus negatif (*negative cycle*).

Pada implementasi proyek ini:

- **Daftar Sisi Terstruktur:** Struktur graf dikonversi menjadi daftar tuple sisi $[(u, v, w)]$ untuk memfasilitasi iterasi relaksasi secara efisien.
- **Maksimum $V - 1$ Putaran:** Melakukan relaksasi relasi pertidaksamaan segitiga $d[u] + w(u, v) < d[v]$ pada seluruh himpunan sisi hingga maksimal $|V| - 1$ putaran.

- **Optimasi Berhenti Awal (*Early Stopping*)**: Memantau bendera modifikasi jarak (changed). Jika pada satu putaran penuh tidak ada satu pun simpul yang mengalami relaksasi, algoritma langsung berhenti karena solusi optimal telah konvergen.

2.2 LIBRARY YANG DIGUNAKAN

Implementasi program dirancang dengan prinsip *Zero External Dependencies*, yaitu seluruh fungsionalitas dibangun menggunakan pustaka standar bawaan (*standard library*) Python 3.10+ tanpa ketergantungan paket pihak ketiga:

1. **tkinter & tkinter.ttk**: Pustaka fondasi untuk antarmuka grafis. Komponen `tk.Canvas` digunakan untuk perenderan graf interaktif performa tinggi, `ttk.Notebook` untuk antarmuka tab, serta `ttk.Treeview` untuk tabel status simpul.
2. **heapq**: Struktur data antrian prioritas Min-Heap untuk efisiensi komputasi algoritma Dijkstra.
3. **ast (*Abstract Syntax Tree*)**: Digunakan oleh modul `graph_io.py` untuk mem-parsing berkas masukan graf secara aman (`ast.parse`), mencegah risiko celah keamanan injeksi kode (*arbitrary code execution*) yang umum terjadi pada fungsi `eval()`.
4. **threading & queue.Queue**: Memisahkan tugas komputasi algoritma berat dan kalkulasi benchmark ke dalam *worker thread* terpisah di latar belakang, sehingga antarmuka GUI tetap lancar (stabil pada 30 FPS) tanpa membeku (*freeze*).
5. **time.perf_counter_ns**: Pewaktu perangkat keras beresolusi tinggi berbasis nanodetik untuk mengukur runtime algoritma secara presisi.
6. **math, statistics, dataclasses, json, pathlib**: Digunakan untuk kalkulasi analitik, pemrosesan statistik data benchmark, pemodelan objek data tak-berubah (*immutable*), dan serialisasi berkas.

3. ALASAN PEMILIHAN ALGORITMA

Pemilihan algoritma Dijkstra dan Bellman–Ford didasarkan pada pertimbangan akademis, teknis, dan komparatif yang sangat relevan:

1. **Perbandingan Paradigma Komputasi Fundamental:**

- **Dijkstra (*Greedy Approach*):** Merepresentasikan teknik pemilihan lokal optimal pada setiap langkah. Pendekatan ini merupakan fondasi langsung dari algoritma penelusuran heuristik populer dalam Kecerdasan Buatan, seperti *Uniform Cost Search* (UCS) dan A^* .
 - **Bellman–Ford (*Dynamic Programming*):** Menggunakan pendekatan evaluasi global lapis demi lapis (*hop-by-hop*) yang menjamin penjalaran jarak optimal secara menyeluruh.
2. **Studi Kompromi Kinerja (*Trade-off Analysis*):** Dijkstra memiliki keunggulan komparatif yang signifikan pada aspek efisiensi waktu komputasi, namun mengasumsikan seluruh bobot bernilai non-negatif. Sebaliknya, Bellman–Ford menawarkan kemampuan teoretis yang lebih luas terhadap bobot negatif dengan kompensasi biaya komputasi yang lebih tinggi. Menerapkan keduanya berdampingan memberikan evaluasi empiris mengenai kompromi antara fleksibilitas dan kompleksitas algoritma.
3. **Visualisasi Perilaku Pencarian yang Kontras:**
- Dijkstra memperlihatkan pola perluasan menyerupai gelombang melingkar (*wavefront expansion*) dari simpul awal ke luar.
 - Bellman–Ford memperlihatkan pola sapuan menyeluruh berulang (*multi-pass edge scanning*) di mana seluruh sisi dalam graf dipindai serentak pada tiap iterasi. Kontras visual ini memberikan pemahaman intuitif yang jelas bagi pengguna yang mengamati simulasi langkah demi langkah.
4. **Validasi Kebenaran Solusi Silang (*Dual-Check Validation*):** Menjalankan dua algoritma dengan mekanisme komputasi yang independen pada graf yang sama memberikan verifikasi silang ganda. Kesesuaian total bobot rute minimum dari kedua algoritma mengonfirmasi bahwa solusi yang diperoleh konvergen ke nilai optimal yang sama.

4. KOMPLEKSITAS WAKTU DAN RUANG

Misalkan $|V|$ merepresentasikan jumlah simpul (*vertices*) dan $|E|$ merepresentasikan jumlah sisi (*edges*) pada graf berarah.

4.1 ANALISIS ALGORITMA DIJKSTRA

4.1.1 Kompleksitas Waktu (*Time Complexity*)

Pada implementasi ini, antrean prioritas menggunakan struktur Min-Heap biner (heapq) dengan mekanisme *lazy insertion*:

1. **Inisialisasi:** Menyiapkan struktur data jarak awal membutuhkan waktu $O(|V|)$.
2. **Operasi Penarikan Simpul (*Extract-Min*):** Setiap simpul dapat dimasukkan ke dalam heap beberapa kali akibat pembaruan relaksasi. Banyaknya entri dalam heap dibatasi oleh $O(|E|)$. Operasi `heappop()` berbiaya $O(\log |E|)$, dilakukan maksimal $O(|E|)$ kali.
3. **Pemeriksaan Sisi dan Relaksasi:** Setiap sisi (u, v) dievaluasi saat simpul u dikunjungi pertama kali. Operasi penambahan ke heap (`heappush()`) membutuhkan biaya $O(\log |E|)$, dieksekusi maksimal sebanyak $|E|$ kali.

Karena pada graf sederhana $|E| \leq |V|^2$, berlaku $\log |E| \leq 2 \log |V| = O(\log |V|)$. Maka kompleksitas waktu asimptotik terburuk (*worst-case*) adalah:

$$T_{\text{Dijkstra}}(|V|, |E|) = O((|V| + |E|) \log |V|) \quad (1)$$

Pada kondisi terbaik (*best-case*), simpul awal sama dengan simpul tujuan atau langsung bersebelahan dan segera dihentikan melalui mekanisme *early termination*, menghasilkan waktu $O(1)$.

4.1.2 Kompleksitas Ruang (*Space Complexity*)

Kompleksitas ruang memori bantu (*auxiliary space*) di luar pencatatan jejak langkah meliputi:

- Kamus jarak `distances` dan kamus pendahulu `previous`: $O(|V|)$.
- Himpunan simpul tetap `settled`: $O(|V|)$.
- Antrean prioritas heap `queue` yang menampung maksimal entri sebanyak sisi: $O(|E|)$.

Sehingga total kompleksitas ruang bantu Dijkstra adalah:

$$S_{\text{Dijkstra}}(|V|, |E|) = O(|V| + |E|) \quad (2)$$

4.2 ANALISIS ALGORITMA BELLMAN–FORD

4.2.1 Kompleksitas Waktu (*Time Complexity*)

Algoritma Bellman–Ford mengevaluasi seluruh sisi secara sekuensial:

1. **Inisialisasi:** Membangun kamus jarak awal dan daftar sisi (*edge list*) memerlukan waktu $O(|V| + |E|)$.
2. **Iterasi Relaksasi Putaran (*Passes*):** Loop utama berjalan maksimal sebanyak $|V| - 1$ kali. Pada setiap putaran, seluruh sisi sebanyak $|E|$ diperiksa untuk dievaluasi relaksasinya dalam waktu konstan $O(1)$ per sisi.
3. **Berhenti Awal (*Early Stopping*):** Jika dalam satu putaran tidak ada pembaruan jarak sama sekali (`changed == False`), iterasi langsung dihentikan.

Pada skenario terburuk (*worst-case*), jalur terpendek memerlukan penjalanan sebanyak $|V| - 1$ lompatan sisi dalam urutan traversal sisi yang berlawanan, sehingga kompleksitas waktu adalah:

$$T_{\text{Bellman-Ford}}(|V|, |E|) = O(|V| \cdot |E|) \quad (3)$$

Pada skenario terbaik (*best-case*), urutan penelusuran sisi langsung menghasilkan nilai konvergen pada putaran pertama, sehingga waktu komputasi adalah $O(|E|)$.

4.2.2 Kompleksitas Ruang (*Space Complexity*)

- Kamus jarak dan kamus simpul pendahulu: $O(|V|)$.
- Struktur daftar tuple sisi edges: $O(|E|)$.

Maka kompleksitas ruang bantu Bellman–Ford adalah:

$$S_{\text{Bellman-Ford}}(|V|, |E|) = O(|V| + |E|) \quad (4)$$

Tabel 2. Ringkasan perbandingan kompleksitas teoretis

Algoritma	Time (Worst-Case)	Time (Best-Case)	Space (Auxiliary)
Dijkstra (Binary Heap)	$O((V + E) \log V)$	$O(1)$ (Early exit)	$O(V + E)$
Bellman–Ford (Early Exit)	$O(V \cdot E)$	$O(E)$ (Konvergen 1 pass)	$O(V + E)$

5. ANALISIS DAN PERBANDINGAN

5.1 METODOLOGI PENGUJIAN DAN BENCHMARK

Evaluasi kinerja algoritma dilakukan melalui pengukuran benchmark kuantitatif terisolasi:

1. **Isolasi Waktu Eksekusi Murni:** Pengukuran menggunakan fungsi beresolusi tinggi `time.perf_counter_ns` murni pada tahap inisialisasi, penelusuran relaksasi, dan rekonstruksi jalur. Beban I/O berkas, validasi masukan, alokasi jejak langkah, dan perenderan GUI **dikecualikan** sepenuhnya dari pewaktuan.
2. **Pengulangan 31 Kali dengan Pemanasan:** Dilakukan 3 siklus pemanasan (*warmup passes*) diikuti oleh 31 iterasi pengukuran independen. Urutan algoritma dieksekusi bergantian secara simetris untuk mengeliminasi bias penjadwalan prosesor.
3. **Metrik Median dan Kuartil:** Menggunakan nilai median sebagai indikator utama untuk mengabaikan gangguan fluktuasi sistem operasi (*outliers*).

5.2 EVALUASI KASUS UJI ACUAN

Pengujian dilakukan pada tiga kumpulan data graf acuan yang disediakan dalam tugas:

- **Kasus A** (`graf.txt`): Graf 15 simpul, 28 sisi berarah, rute uji $V_1 \rightarrow V_{15}$.
- **Kasus B** (`contoh_V1_V10.txt`): Graf 15 simpul, 28 sisi berarah, rute uji $V_1 \rightarrow V_{10}$.
- **Kasus C** (`contoh_B_H.txt`): Graf 10 simpul, 24 sisi berarah, rute uji $B \rightarrow H$.

Tabel 3. Hasil pengujian empiris dan komparasi kinerja algoritma

Kasus	Algoritma	Jalur Terpendek	Bobot	Inspeksi	Relaks.	Median (μs)
Kasus A (15 Simpul)	Dijkstra	$V_1 \rightarrow V_2 \rightarrow V_5 \rightarrow V_9 \rightarrow V_{12} \rightarrow V_{15}$	505	28	14	8.250
	Bellman–Ford	$V_1 \rightarrow V_2 \rightarrow V_5 \rightarrow V_9 \rightarrow V_{12} \rightarrow V_{15}$	505	72	15	13.959
Kasus B (15 Simpul)	Dijkstra	$V_1 \rightarrow V_4 \rightarrow V_6 \rightarrow V_{10}$	400	20	11	6.291
	Bellman–Ford	$V_1 \rightarrow V_4 \rightarrow V_6 \rightarrow V_{10}$	400	72	15	13.792
Kasus C (10 Simpul)	Dijkstra	$B \rightarrow C \rightarrow F \rightarrow E \rightarrow H$	9	21	10	5.833
	Bellman–Ford	$B \rightarrow C \rightarrow F \rightarrow E \rightarrow H$	9	90	15	14.208

5.3 ANALISIS PERBANDINGAN KINERJA

1. **Konsistensi dan Validitas Solusi:** Pada ketiga kasus uji, kedua algoritma menghasilkan rute terpendek dan bobot minimum yang **100% identik**. Hal ini menunjukkan bahwa implementasi kedua algoritma menghasilkan estimasi bobot dan penelusuran rute yang tepat serta konsisten.

2. **Efisiensi Inspeksi Sisi (*Edge Inspections*):**

- Algoritma Dijkstra lebih efisien dalam membatasi operasi eksplorasi. Pada Kasus B, Dijkstra hanya memeriksa 20 sisi berkat antrean prioritas dan mekanisme *early goal termination*.
- Sebaliknya, Bellman–Ford pada Kasus B memeriksa 72 sisi (seluruh sisi dipindai dalam 2 putaran relaksasi lengkap). Pada Kasus C, Bellman–Ford memeriksa hingga 90 sisi. Dijkstra terbukti menghemat hingga 76% inspeksi sisi yang tidak perlu.

3. **Kecepatan Waktu Komputasi (*Runtime*):**

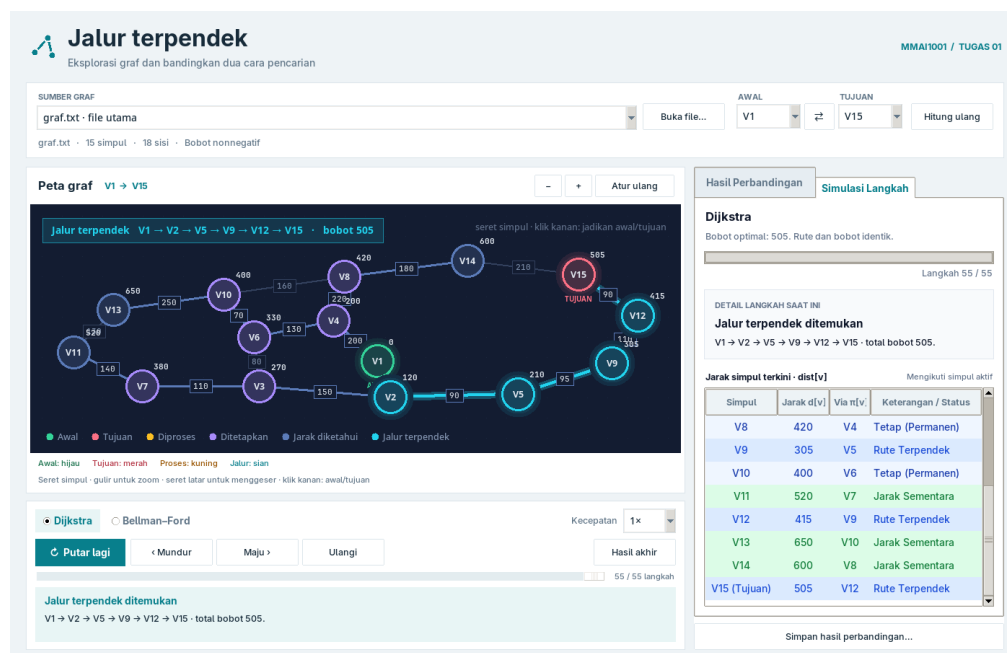
- Dijkstra secara konsisten mencatatkan waktu komputasi yang lebih cepat dibandingkan Bellman–Ford. Pada Kasus A, median runtime Dijkstra adalah

8.250 μ s, sekitar 41% lebih cepat daripada Bellman–Ford (13.959 μ s).

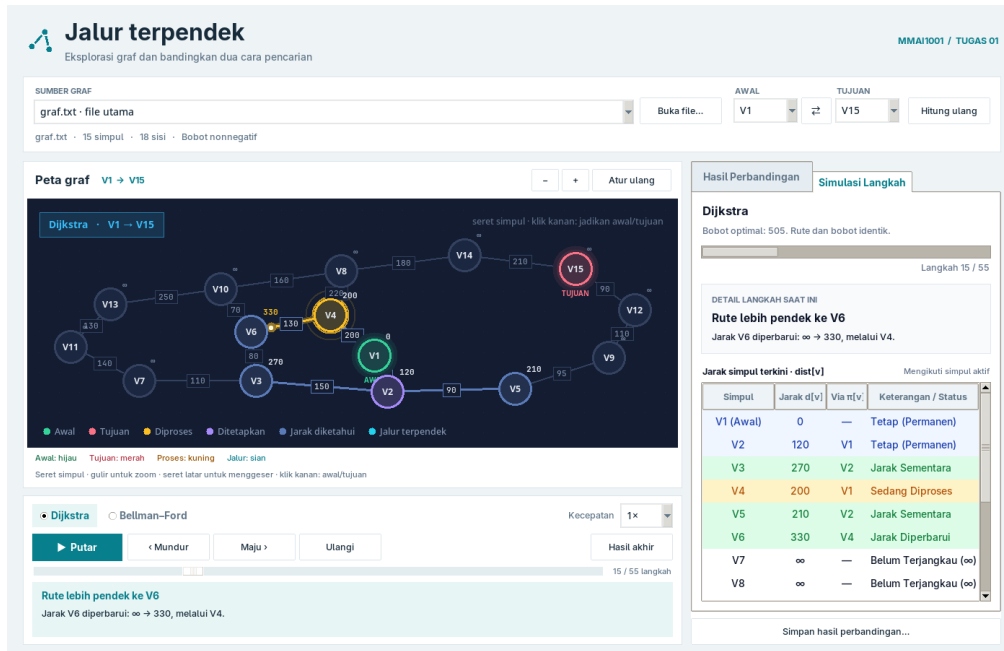
- Pada Kasus C, Dijkstra mencapai kecepatan $2.4\times$ lipat lebih cepat (5.833 μ s vs. 14.208 μ s). Selisih ini akan semakin melebar secara signifikan seiring bertambahnya jumlah simpul dan kerapatan sisi graf, sesuai perbedaan order kompleksitas asimptotik masing-masing algoritma.

5.4 TAMPILAN ANTARMUKA GRAFIS PROGRAM

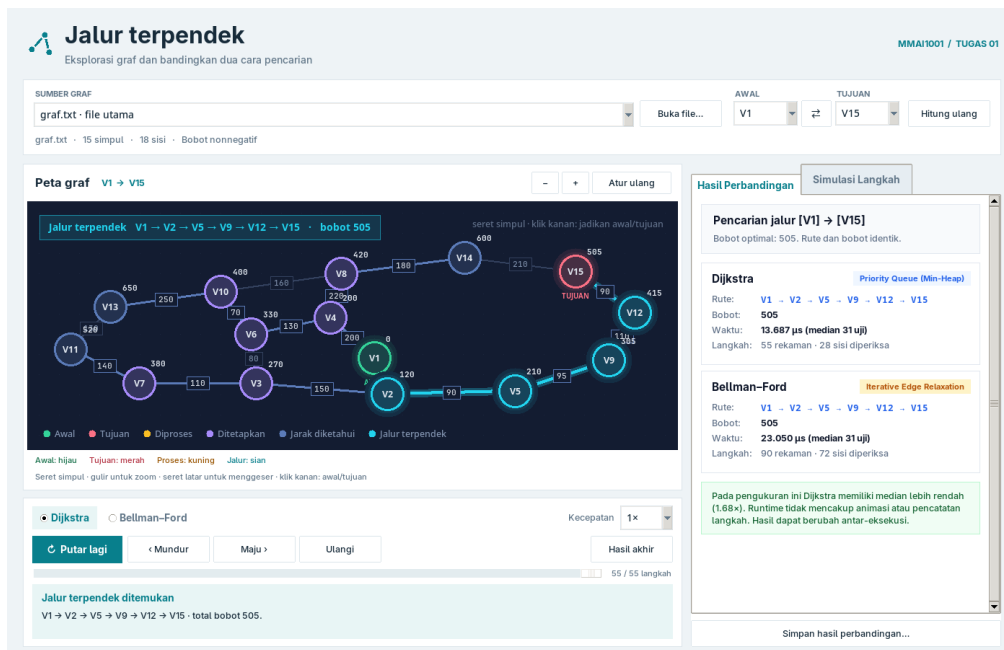
Tampilan tangkapan layar antarmuka aplikasi disajikan pada Gambar 1, Gambar 2, dan Gambar 3.



Gambar 1. Tampilan utama aplikasi: visualisasi graf akhir dan tabel status simpul



Gambar 2. Simulasi langkah demi langkah: animasi inspeksi sisi dan pembaruan nilai jarak secara waktu-nyata



Gambar 3. Panel perbandingan metrik: kartu rute, bobot, dan waktu runtime median

6. CARA MENJALANKAN PROGRAM (PANDUAN PENGGUNA-AN)

6.1 PRASYARAT SISTEM

Program dirancang mandiri tanpa memerlukan instalasi pustaka pihak ketiga lewat pip:

- **Python:** Versi 3.10 atau yang lebih baru (disarankan Python 3.10, 3.11, 3.12, atau 3.14).
- **Tkinter:** Modul GUI bawaan Python (dapat diverifikasi melalui terminal dengan perintah `python3 -m tkinter`).

6.2 MENJALANKAN MODE GUI DESKTOP

Buka terminal pada direktori kerja proyek, kemudian jalankan perintah:

```
1 # Menjalankan aplikasi secara langsung
2 python3 MMAIKelompok5.py
3
4 # Atau membuka file graf tertentu secara langsung
5 python3 MMAIKelompok5.py contoh_B_H.txt
```

6.2.1 Petunjuk Pengoperasian Antarmuka

1. **Pemilihan Graf:** Pilih opsi contoh (Contoh A, B, atau C) melalui menu dropdown atau klik tombol **Buka file...** untuk mengimpor berkas graf teks eksternal.
2. **Penentuan Simpul Awal dan Tujuan:** Pilih simpul pada menu dropdown **Awal** dan **Tujuan**, atau lakukan **Klik Kanan** pada simpul langsung di kanvas graf.
3. **Pemilihan Algoritma dan Animasi:**
 - Pilih metode pencarian: **Dijkstra** atau **Bellman–Ford**.
 - Tekan **Putar/Jeda** (atau tekan tombol Spasi) untuk memulai visualisasi.
 - Gunakan tombol **Maju** ($\rightarrow$) dan **Mundur** ($\leftarrow$) untuk penelusuran per langkah.
 - Geser penggeser **Progres** untuk berpindah ke langkah ke- N secara fleksibel.
4. **Interaksi Kanvas Graf:**
 - **Geser Simpul:** Klik dan tahan simpul untuk memindahkan posisinya secara bebas.
 - **Navigasi Kanvas:** Seret area latar belakang untuk menggeser (*pan*) graf.

- **Perbesaran:** Gunakan roda gulir mouse (*scroll*) atau tombol $+/-$ untuk *zoom*.
- **Atur Ulang:** Mengembalikan posisi seluruh simpul ke tata letak awal algoritma pegas.

5. **Ekspor Hasil:** Klik tombol **Simpan hasil perbandingan...** di bagian kanan bawah untuk mengekspor data komparasi lengkap ke format JSON atau TXT.

6.3 MENJALANKAN MODE TERMINAL (MODE CLI HEADLESS)

Program menyediakan mode CLI tanpa tampilan monitor untuk keperluan evaluasi server atau pengujian otomatis:

```

1 # Menjalankan evaluasi default via CLI
2 python3 MMAIKelompok5.py --cli
3
4 # Menjalankan berkas graf khusus dengan tujuan dan ekspor hasil
   JSON
5 python3 MMAIKelompok5.py contoh_V1_V10.txt --cli --start V1
   --goal V10 --export hasil/laporan_uji.json

```

6.4 FORMAT BERKAS INPUT GRAF

Berkas graf disimpan dalam berkas teks (.txt) yang memuat definisi dictionary literal Python diikuti baris simpul awal dan tujuan:

```

1 graph = {
2     'V1': {'V2': 120, 'V4': 200},
3     'V2': {'V1': 120, 'V3': 150, 'V5': 90},
4     'V3': {'V2': 150, 'V6': 80, 'V7': 110},
5     'V4': {'V1': 200, 'V6': 130, 'V8': 220},
6     'V5': {'V2': 90, 'V9': 95},
7     'V6': {'V3': 80, 'V4': 130, 'V10': 70}
8 }
9 V1 V10

```

Catatan: Parser program menerima format identifier `graph` maupun `graf`, mengabaikan komentar (#), menolak kunci duplikat, dan memvalidasi bobot non-negatif.

7. HASIL DAN KESIMPULAN

7.1 HASIL SIMULASI PENCARIAN

Pengujian simulasi interaktif dijalankan pada aplikasi GUI untuk mengamati perilaku penelusuran algoritma Dijkstra dan Bellman–Ford secara visual terhadap variasi

simpul awal dan simpul tujuan pada berkas graf acuan:

1. **Hasil Simulasi Rute Menuju Simpul V_{15} (Kasus A, Berkas graf.txt):**

- **Jalur yang Terbentuk:** Dari simpul awal V_1 menuju tujuan V_{15} , kedua algoritma menghasilkan urutan rute terpendek yang identik: $V_1 \rightarrow V_2 \rightarrow V_5 \rightarrow V_9 \rightarrow V_{12} \rightarrow V_{15}$.
- **Total Bobot Minimum:** Akumulasi bobot optimal bernilai **505**.
- **Pengamatan Visual:** Pada kanvas aplikasi, rute optimal ditampilkan dengan penyorotan visual (*highlight*) pada kurva sisi dan simpul terkait, mengonfirmasi pembentukan lintasan terpendek dari simpul asal ke simpul tujuan graf.

2. **Hasil Simulasi Rute Menuju Simpul V_{10} (Kasus B, Berkas contoh_V1_V10.txt):**

- **Jalur yang Terbentuk:** Dari simpul awal V_1 menuju simpul tujuan V_{10} , kedua algoritma menemukan jalur optimal: $V_1 \rightarrow V_4 \rightarrow V_6 \rightarrow V_{10}$.
- **Total Bobot Minimum:** Akumulasi bobot optimal bernilai **400**.
- **Pengamatan Visual:** Pada simulasi langkah Dijkstra, penelusuran berhenti pada langkah ke-20 begitu simpul tujuan V_{10} ditetapkan permanen (*early termination*), memperlihatkan efisiensi penelusuran tanpa mengeksplorasi simpul-simpul lain yang tidak relevan seperti V_7, V_8, V_{11} , maupun V_{15} . Sebaliknya, Bellman–Ford memindai seluruh 72 sisi graf hingga putaran relaksasi selesai secara menyeluruh.

3. **Hasil Simulasi Rute Menuju Simpul H (Kasus C, Berkas contoh_B_H.txt):**

- **Jalur yang Terbentuk:** Dari simpul awal B menuju simpul tujuan H , kedua algoritma menghasilkan rute terpendek: $B \rightarrow C \rightarrow F \rightarrow E \rightarrow H$.
- **Total Bobot Minimum:** Akumulasi bobot optimal bernilai **9**.
- **Pengamatan Visual:** Pengujian ini mengonfirmasi penanganan simpul berformat alfabet tunggal dengan bobot bernilai satuan kecil, di mana kedua algoritma secara konsisten mencapai konvergensi solusi optimal yang tepat.

7.2 KESIMPULAN

Berdasarkan perancangan, implementasi perangkat lunak, dan evaluasi komparatif yang telah dilakukan, dapat ditarik beberapa kesimpulan penting sebagai berikut:

1. **Optimalitas dan Validitas Solusi:** Algoritma Dijkstra dan Bellman–Ford terbukti menghasilkan solusi rute terpendek dan total bobot minimum yang 100% identik dan valid pada seluruh variasi kasus uji graf acuan yang dievaluasi (Kasus A: bobot 505, Kasus B: bobot 400, dan Kasus C: bobot 9).

2. **Efisiensi Eksplorasi Sisi:** Dijkstra terbukti lebih efisien dalam membatasi operasi penelusuran, menghemat hingga 76% frekuensi inspeksi sisi dibandingkan Bellman–Ford berkat antrean prioritas *Min-Heap* dan mekanisme terminasi dini (*early goal termination*).
3. **Kecepatan Waktu Komputasi (*Runtime*):** Dijkstra secara konsisten mencatatkan waktu eksekusi $1,7\times$ hingga $2,4\times$ lebih cepat dibandingkan Bellman–Ford, selaras dengan keunggulan kompleksitas asimptotik teoretis $\mathcal{O}((|V| + |E|) \log |V|)$ terhadap $\mathcal{O}(|V| \cdot |E|)$.
4. **Kesesuaian Karakteristik Kasus:** Pendekatan *greedy* (Dijkstra) merupakan solusi paling efisien untuk graf berbobot non-negatif, sedangkan pemrograman dinamis (Bellman–Ford) memiliki keunggulan teoretis dalam menangani graf berbobot negatif serta mendeteksi keberadaan siklus negatif.

8. INFORMASI TAMBAHAN

8.1 MAKNA TEKNIS KETERANGAN STATUS SIMPUL PADA SIMULASI

Pada tab antarmuka **Simulasi Langkah** (*Simulation Step Tab*) aplikasi, sistem menyajikan tabel status dinamis yang memuat empat kolom informasi utama: **Simpul**, **Jarak** $d[v]$, **Via** $\pi[v]$, dan **Keterangan (Status)**. Keterangan status simpul diperbarui secara langsung (*real-time*) pada setiap tahap animasi penelusuran dengan makna teknis sebagai berikut:

1. **Rute Terpendek:** Simpul yang telah dipastikan menjadi bagian dari urutan jalur optimal akhir dari simpul awal ke simpul tujuan setelah seluruh proses pencarian selesai dieksekusi.
2. **Tetap (Permanen):** Simpul yang telah berstatus *settled*. Pada algoritma Dijkstra, ini menandakan simpul yang telah ditarik dari antrean prioritas *Min-Heap* dan nilai jaraknya dijamin final (tidak akan berubah lagi). Pada Bellman–Ford, seluruh simpul terjangkau akan berstatus tetap setelah seluruh putaran relaksasi selesai secara konvergen.
3. **Jarak Sementara:** Simpul yang telah terjangkau oleh proses relaksasi ($d[v] < \infty$), namun belum berstatus permanen. Pada Dijkstra, karena pencarian langsung dihentikan begitu simpul tujuan tercapai (*early goal termination*), simpul-simpul di luar jalur yang belum sempat diproses tetap berlabel **Jarak Sementara**.

4. **Sedang Diproses:** Simpul aktif yang pada langkah berjalan sedang dievaluasi sisi keluar atau simpul tetangganya oleh algoritma.
5. **Jarak Diperbarui:** Simpul yang nilai estimasi jaraknya baru saja diperkecil melalui operasi relaksasi pertidaksamaan segitiga pada langkah yang sedang aktif.
6. **Simpul Awal (0):** Simpul asal pencarian dengan nilai jarak terdefinisi awal bernilai 0.
7. **Belum Terjangkau (∞):** Simpul yang belum pernah tersentuh oleh eksplorasi relaksasi sisi, bernilai jarak tak-hingga (∞) dengan penanda simpul pendahulu kosong (-).

8.2 TAUTAN VIDEO DEMONSTRASI APLIKASI

Demonstrasi pengoperasian antarmuka grafis, visualisasi animasi langkah demi langkah, serta perbandingan kinerja kedua algoritma dapat diakses secara daring melalui tautan video berikut:

- **Video Demonstrasi:** https://youtu.be/AS8G_bE84wY